AIM- To implement a super loop–based embedded program that sequentially performs LED blinking, button status reading, and sensor data acquisition using simple timing counters.

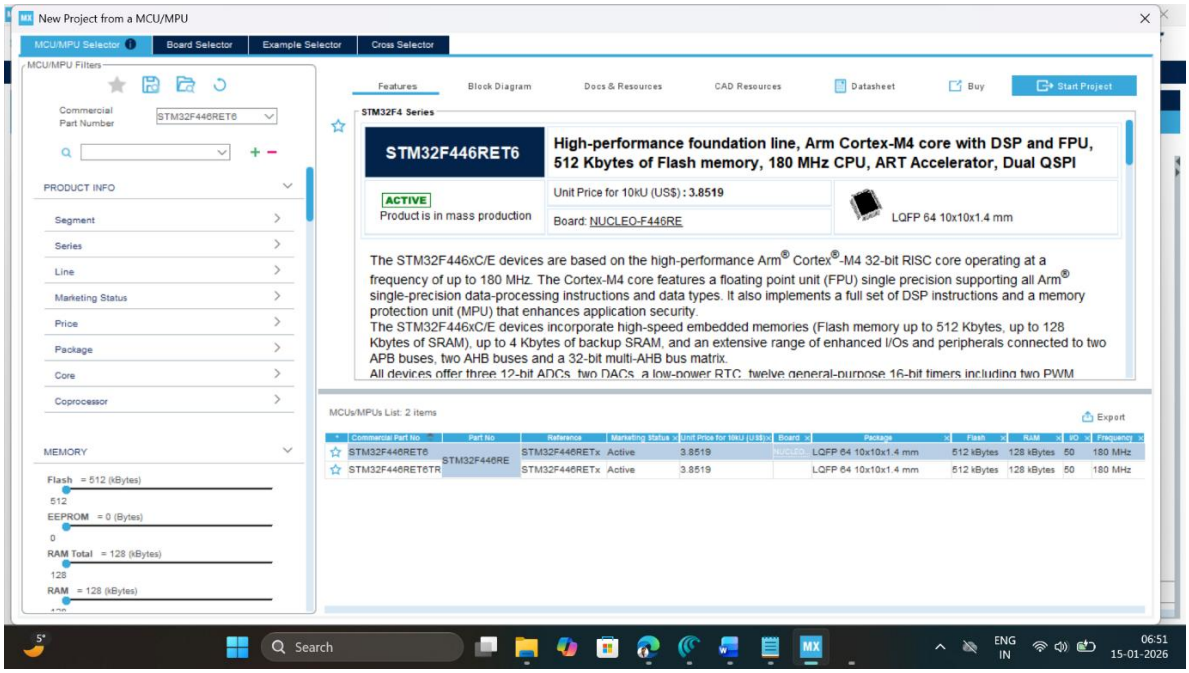
APPARATUS- STM32 Nucleo-F446RE development board, Infrared sensor, jumper cables, USB Type-A to Mini-B cable, STM32 CUBE IDE, STM32 CubeMX.

THEORY

The term super loop is simply an infinite `while(1)` loop that repeatedly executes a fixed set of tasks—here, LED blinking, button reading, and IR sensor sampling—in a defined order. The super loop is part of a foreground–background architecture: interrupts such as SysTick form the foreground and provide a regular time base, while the background loop runs the main application code and uses the tick count (via `HAL_GetTick()`) to decide when each task should run again. Instead of blocking delays, the program maintains separate software counters for each task (e.g. LED, button, IR sensor), increments them using the elapsed tick time, and when a counter reaches its preset period (such as 500 ms for the LED or 200 ms for the ADC) the corresponding task code is executed once and the counter is reset. In this way multiple periodic activities share a single CPU context cooperatively: all tasks run sequentially in the loop, but because each task is written to be short and non-blocking, the loop iterates quickly and each task appears to run "in parallel" at its own rate without the complexity of an RTOS or priority-based scheduler.

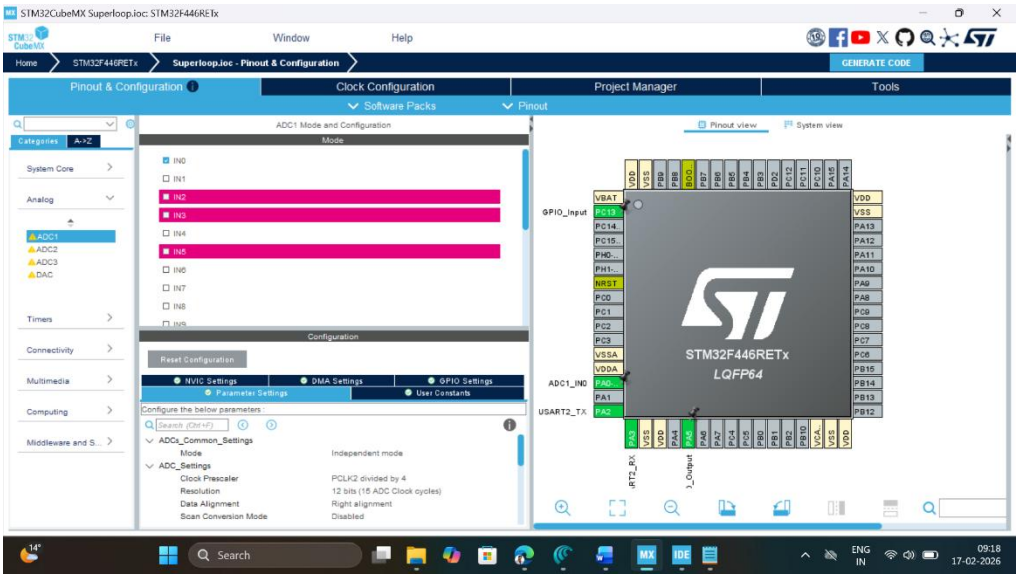
PROCEDURE

1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.

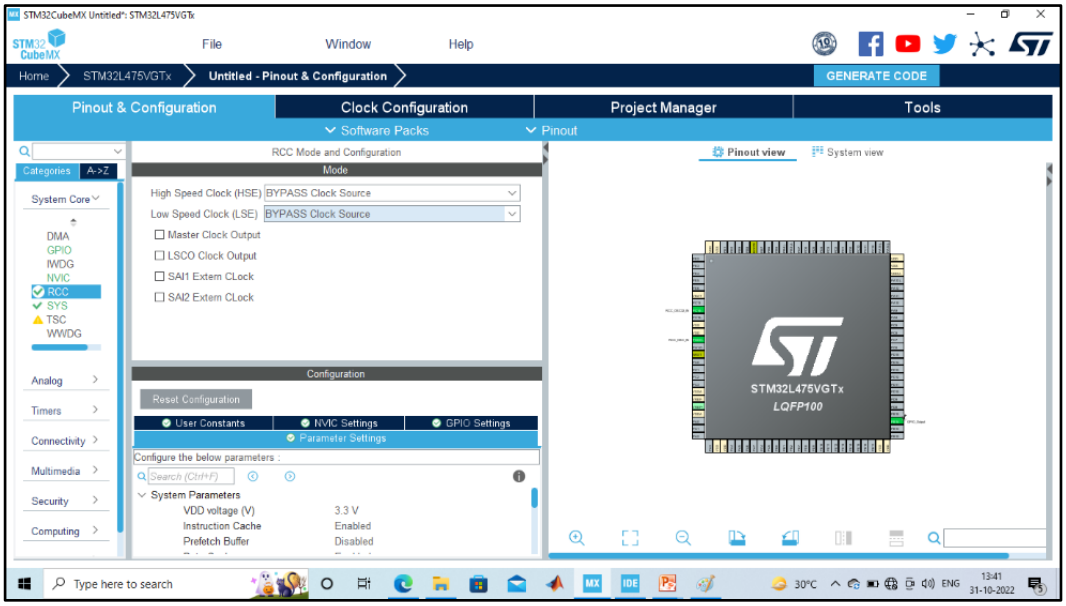


2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L446RE.
3. In system Core select GPIO and right click on pin PA5 to make it as GPIO output pin as shown below as LED (Inbuilt LED) and PC13 as Pushbutton (USER) – PULL UP

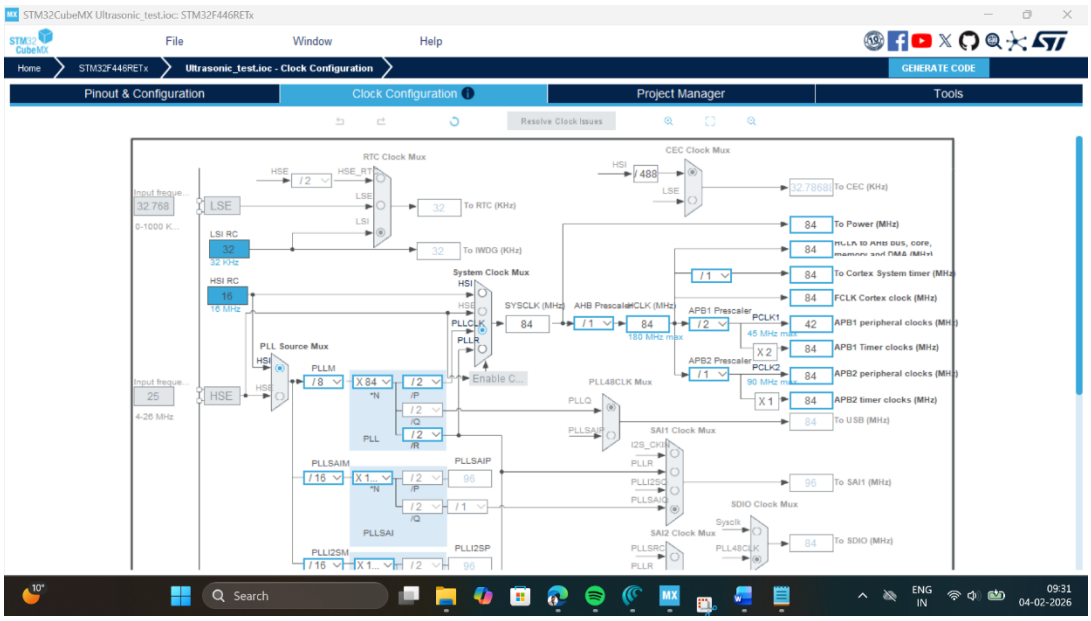
mode relates to it in the board. Furthermore, initialize PA0 (ADC1_IN0) to read IR sensor as Analog input.



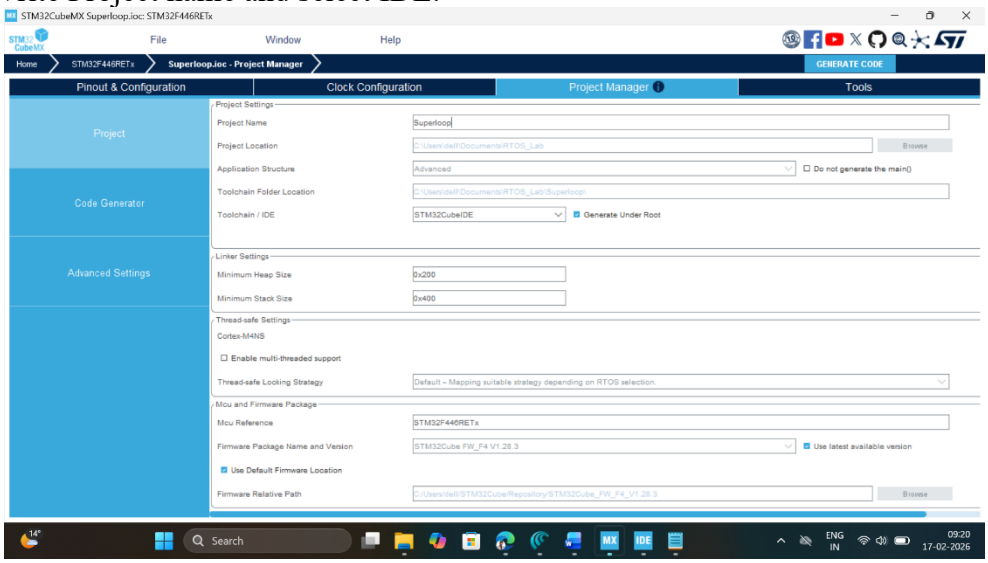
4. **Clock Selection**: Select the internal clock by clicking RCC and selecting BYPASS clock source



5. **Clock Configuration:** Configure clock as shown below to get maximum internal clock frequency of 84MHz.



6. Write Project name and select IDE.

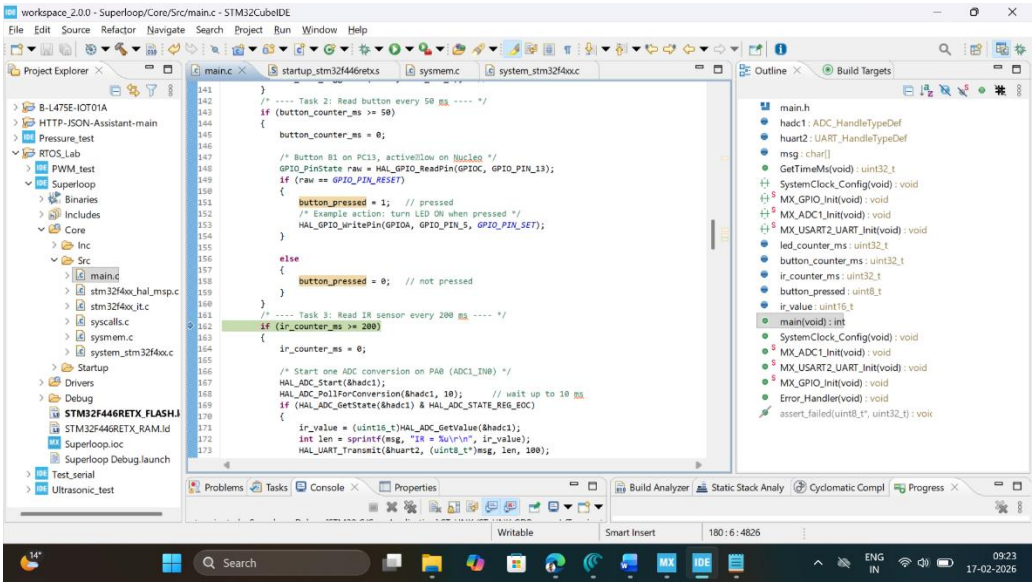


7. Click on generate code.
8. Click on open project.
9. Click ‘OK’ on the generated popup showing “successfully imported the project on the workspace” and the project will be successfully added to the cube IDE.
10. Open the main source code on Cube IDE by clicking on **main.c**
11. To generate bin file and hex file, right click on project and click properties.
12. Expand C/C++ Build, click on setting, click on MCU post build outputs and select convert to binary file as well as convert to hex file and click “Apply and Close ".
13. After configuration write following instructions in the **main.c** file and compile the program and ensure there will be zero error after compiling.

```
/* ---------- Global handles (from CubeMX) ---------- */
ADC_HandleTypeDef hadc1;

/* ---------- Global variables (easy to watch in debugger) ---------- */
volatile uint32_t led_counter_ms    = 0;
volatile uint32_t button_counter_ms = 0;
```

```
volatile uint32_t ir_counter_ms     = 0;
volatile uint32_t last_time_ms      = 0;

volatile uint8_t  button_pressed = 0;
volatile uint16_t ir_value       = 0;   // IR sensor raw ADC value (0..4095)


/* ---------- Simple time helper ---------- */
static inline uint32_t GetTimeMs(void)
{
   return HAL_GetTick();   // 1 ms tick from SysTick
}

int main(void)
{
   /* 1. Basic initialization */
   HAL_Init();
   SystemClock_Config();
   MX_GPIO_Init();
   MX_ADC1_Init();

   /* 2. Initialize timing */
   last_time_ms = GetTimeMs();

   while (1)
   {
      /* ---- 2.1 Update time and counters ---- */
      uint32_t now_ms = GetTimeMs();
      uint32_t dt     = now_ms - last_time_ms;
      last_time_ms    = now_ms;

      led_counter_ms    += dt;
      button_counter_ms += dt;
      ir_counter_ms     += dt;

      /* ---- Task 1: Blink LED every 500 ms ---- */
      if (led_counter_ms >= 500)
      {
         led_counter_ms = 0;
         HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);  // PA5: user LED LD2
      }

      /* ---- Task 2: Read button every 50 ms ---- */
      if (button_counter_ms >= 50)
      {
         button_counter_ms = 0;

         // User button B1 on PC13, active-low on Nucleo-64 [web:40]
         GPIO_PinState raw = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);
         if (raw == GPIO_PIN_RESET)
         {
            button_pressed = 1;  // pressed
            // Example action: force LED ON when button pressed
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
         }
         else
         {
            button_pressed = 0;  // not pressed
         }
      }
```

```
        /* ---- Task 3: Read IR sensor every 200 ms ---- */
        if (ir_counter_ms >= 200)
        {
            ir_counter_ms = 0;

            // One ADC conversion on PA0 (ADC1_IN0) [web:112][web:17]
            HAL_ADC_Start(&hadc1);
            HAL_ADC_PollForConversion(&hadc1, 10);      // wait up to 10 ms
            if (HAL_ADC_GetState(&hadc1) & HAL_ADC_STATE_REG_EOC)
            {
                ir_value = (uint16_t)HAL_ADC_GetValue(&hadc1);
                // ir_value now reflects IR sensor voltage (0..3.3 V)
            }
            HAL_ADC_Stop(&hadc1);
        }

        // No HAL_Delay() here: timing is via software counters
    }
}
```



14. To run this program, connect the board with USB cable, click on build, click ‘ok’ and click ‘switch’ on popups.
15. Now you are in debugging mode, click ‘Resume’ to execute the program.

OBSERVATION:

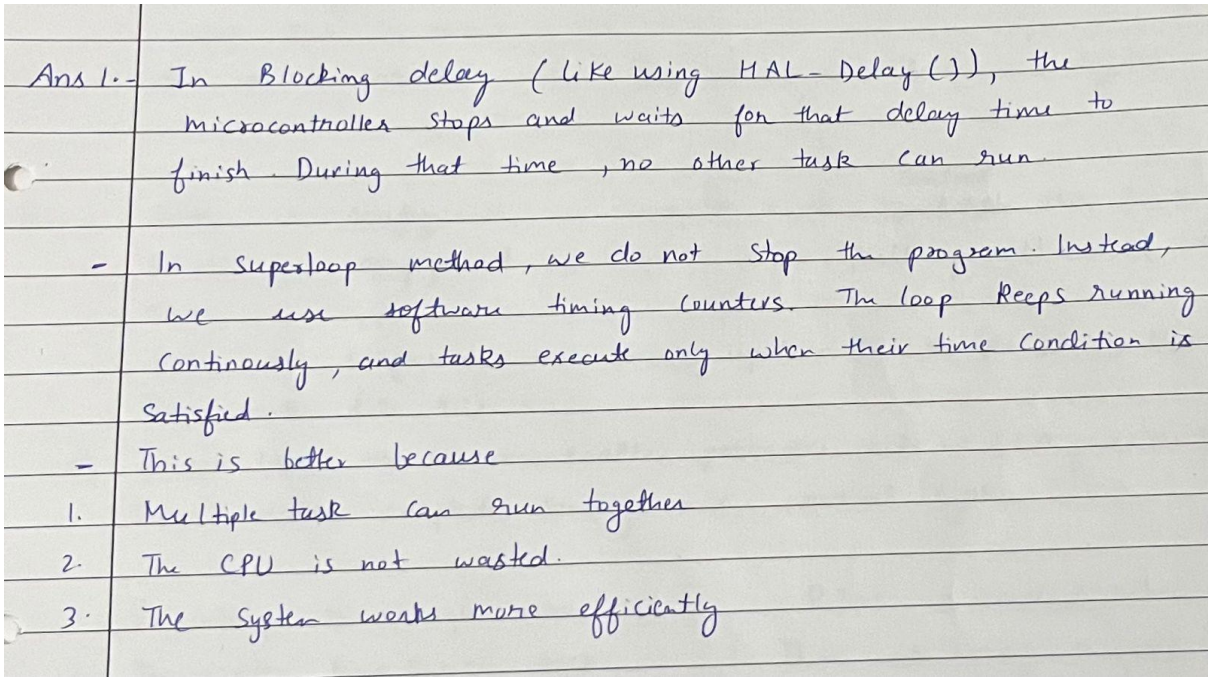
S. No	Status	Remarks
1	**LED**	The LED connected to PA5 toggled every 500 ms (approximately 1 second ON-OFF cycle). It blinked continuously without affecting other tasks.
2	**Button Press**	When the push button (PC13) was pressed, the system detect it correctly (active LOW). The LED turned ON immediately when the button was pressed.
3	**IR_Senssor**	The IR sensor connected to PA0 produced ADC values betw 0–4095. The readings changed according to object distance a surface. Closer objects gave higher ADC values.

RESULT

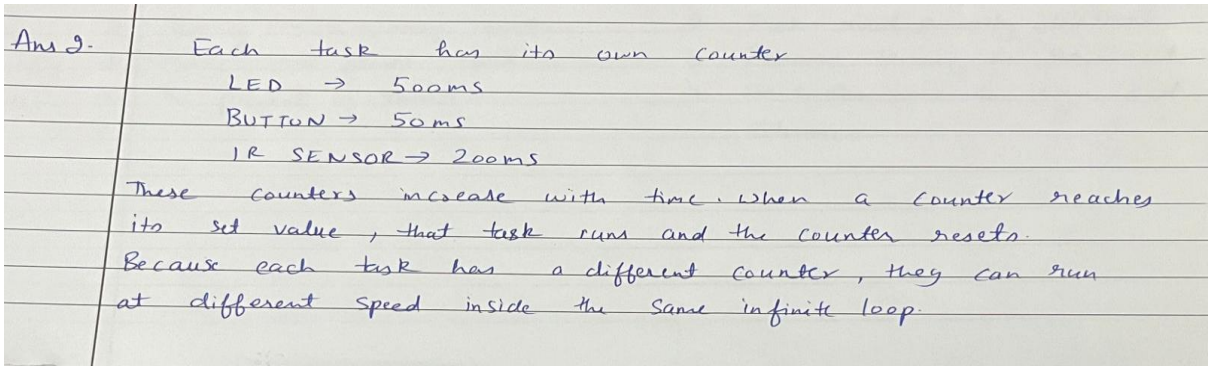
In this experiment the super-loop program on the STM32F446RE successfully executed three periodic tasks—LED blinking, button status reading, and IR sensor data acquisition—within a single infinite loop using non-blocking software timers derived from the SysTick millisecond tick. Students observed that the LED toggled at the expected interval, the button state was correctly detected despite sharing the loop with other activities, and the IR sensor produced varying ADC codes that changed predictably with object distance or surface characteristics, confirming correct GPIO and ADC configuration and demonstrating how multiple simple tasks can run cooperatively in a resource-constrained embedded system without an RTOS.

Reflection Summary

1. How does the super-loop approach used in this experiment differ from using blocking delays, and why is it better suited for running multiple periodic tasks on a simple microcontroller system?

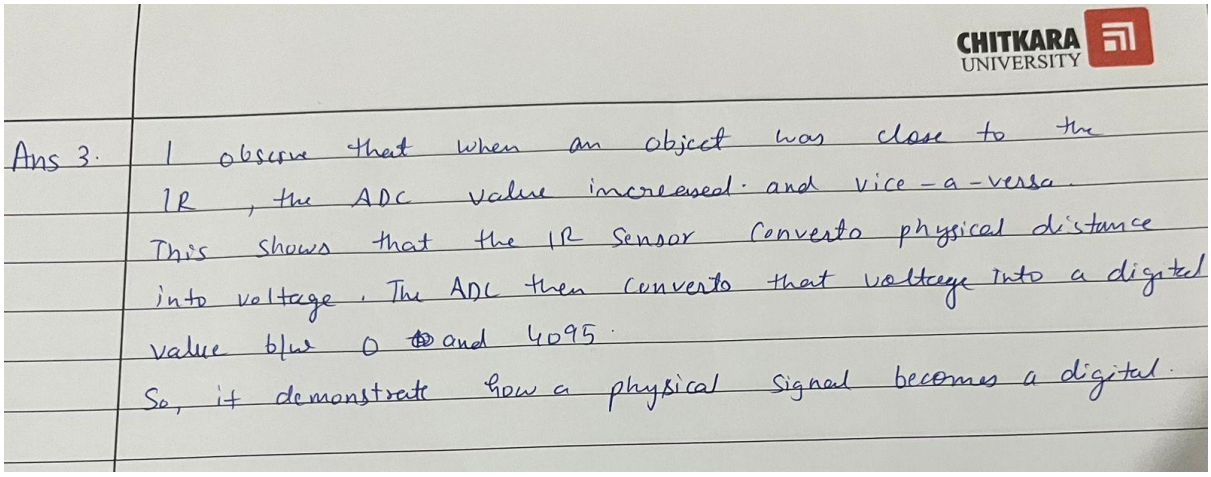


2. In what way do the separate software counters for LED blinking, button sampling, and IR sensor sampling help you schedule these tasks at different rates within a single infinite loop?



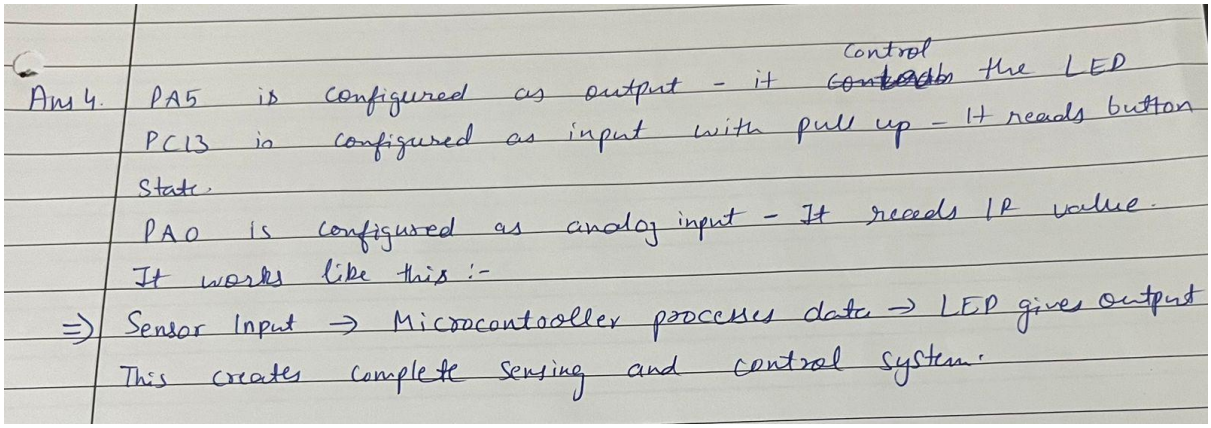
Ans 2. Each task has its own counter
LED → 500ms
BUTTON → 50ms
IR SENSOR → 200ms
These counters increase with time. When a counter reaches its set value, that task runs and the counter resets.
Because each task has a different counter, they can run at different speed inside the same infinite loop.

3. What did you observe about the IR sensor's ADC readings as the distance and surface of the target object changed, and how does this illustrate the conversion of a physical quantity into a digital signal?



Ans 3. I observe that when an object was close to the IR, the ADC value increased. and vice-a-versa.
This shows that the IR sensor converts physical distance into voltage. The ADC then converts that voltage into a digital value b/w 0 and 4095.
So, it demonstrate how a physical signal becomes a digital.

4. How do the configurations of the GPIO pins (PA5 as output, PC13 as input with pull-up) and the ADC channel (PA0 as analog input) work together to support the complete sensing-and-actuation behavior of this system?



Ans 4. PA5 is configured as output - it control the LED
PC13 is configured as input with pull up - It reads button state.
PA0 is configured as analog input - It reads IR value.
It works like this :-
⇒ Sensor Input → Microcontroller processes data → LED gives output
This creates complete sensing and control system.

5. Based on your experience in this lab, under what conditions would a super-loop architecture be sufficient for an embedded application, and when might you need to move to an RTOS-based or more advanced event-driven design?

Superloop is enough when :-
1. The system has simple task.
2. Timing is not very strict.
3. There are fewer task.
4. The project is small.

RTOS is needed when
1. There are many complex tasks.
2. Task need priority control
3. Precise timing is required.
4. The system becomes large and advanced.